

2. (a)

```
LSV-PA-2026-fall > lsv > pa1 > ≡ mul.blif
1  # 2-bit Multiplier
2  .model mul
3  .inputs a1 a0 b1 b0
4  .outputs y3 y2 y1 y0
5
6  #  $y_0 = a_0 * b_0$ 
7  .names a0 b0 y0
8  11 1
9
10 #  $y_1, c_2 = (a_0 * b_1) + (a_1 * b_0)$ 
11 .names a0 b1 p01
12 11 1
13 .names a1 b0 p10
14 11 1
15 .subckt half_adder a=p01 b=p10 s=y1 cout=c2
16
17 #  $y_2, y_3 = (a_1 * b_1) + c_2$ 
18 .names a1 b1 p11
19 11 1
20 .subckt half_adder a=p11 b=c2 s=y2 cout=y3
21 .end
22
23 # Sub-module: 1-bit Half Adder
24 .model half_adder
25 .inputs a b
26 .outputs s cout
27
28 .names a b s
29 10 1
30 01 1
31 .names a b cout
32 11 1
33 .end
```

2. (b)

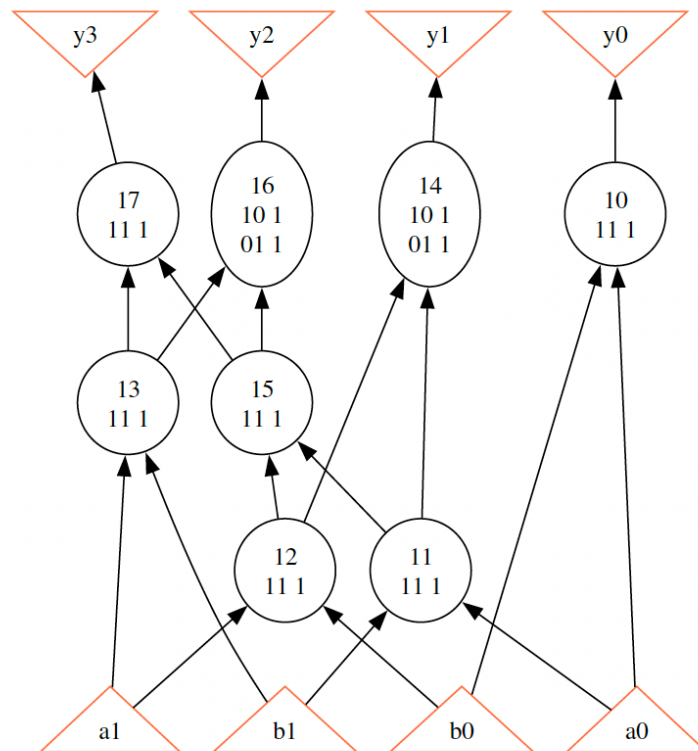
1. 2. 3.

```
abc 01> r lsv/pa1/mul.blif; ps; show;
```

```
abc 01> r lsv/pa1/mul.blif; ps; show;  
Hierarchy reader flattened 2 instances of logic boxes and left 0 black boxes.  
mul : i/o = 4/ 4 lat = 0 nd = 8 edge = 16 cube = 10 lev = 3
```

Network structure visualized by ABC
Benchmark "mul". Time was Fri Sep 25 20:44:53 2026.

The network contains 8 logic nodes and 0 latches.

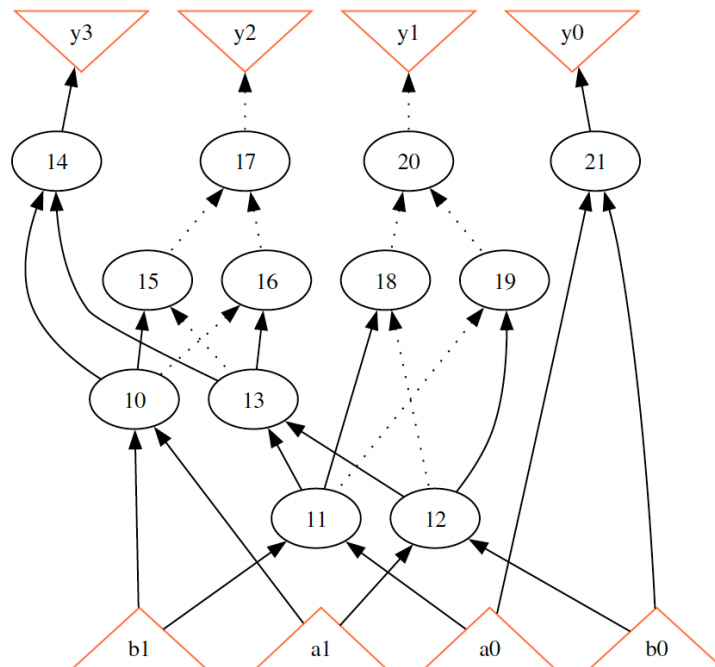


4. 5.

```
abc 02> strash; show;
```

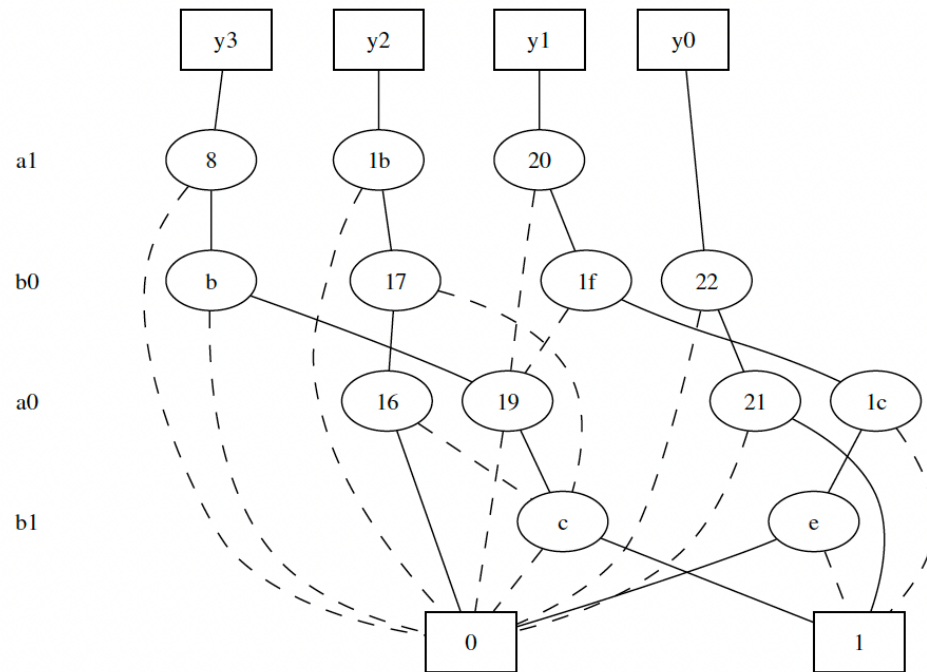
Network structure visualized by ABC
Benchmark "mul". Time was Fri Sep 25 20:47:48 2026.

The network contains 12 logic nodes and 0 latches.



6. 7.

abc 03> collapse; show_bdd -g;

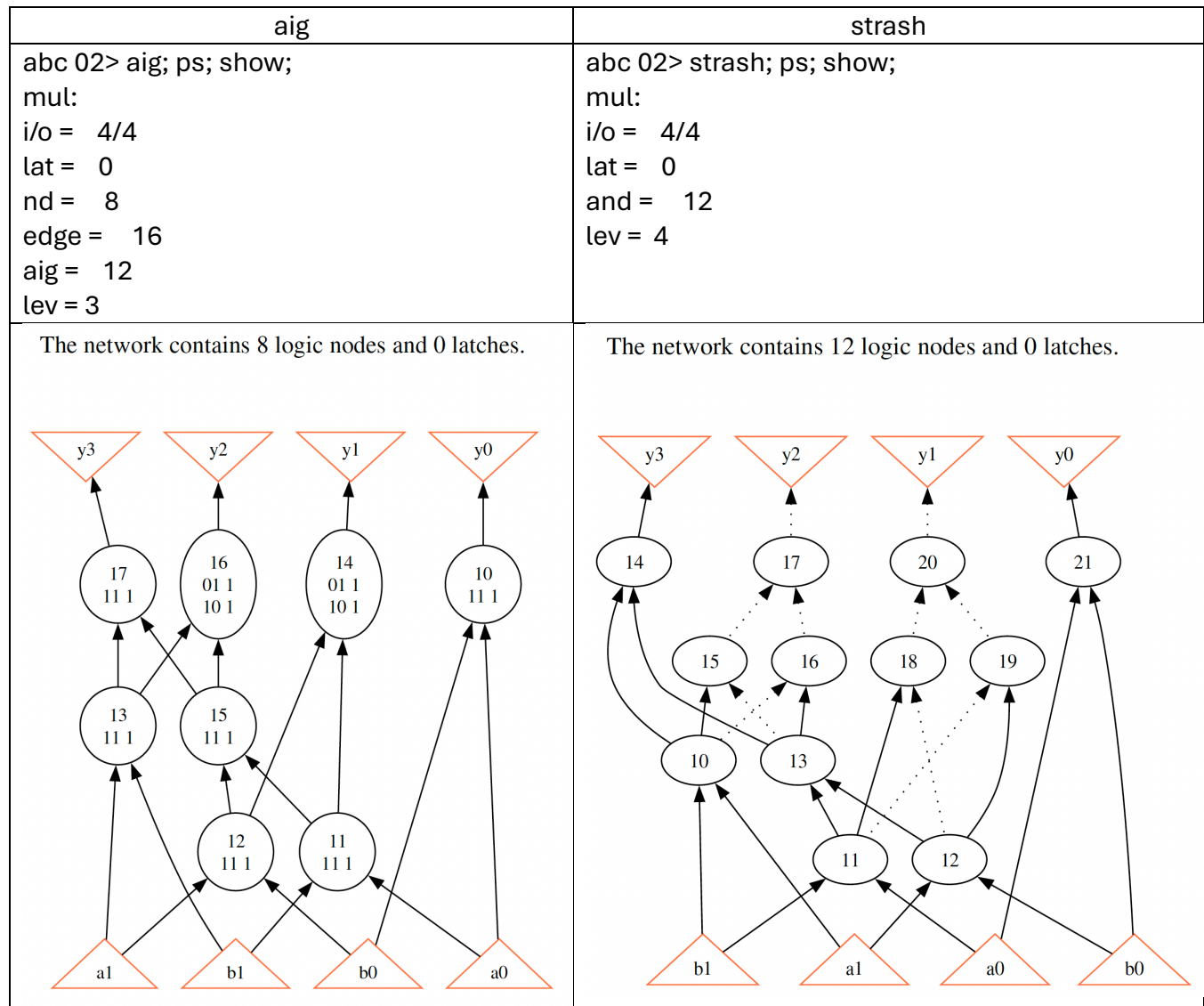


3. (a)

1.

```
abc 02> aig; ps; show;
mul      : i/o = 4/ 4 lat = 0 nd = 8 edge = 16 aig = 12 lev = 3
```

```
abc 02> strash; ps; show;
mul      : i/o = 4/ 4 lat = 0 and = 12 lev = 4
```



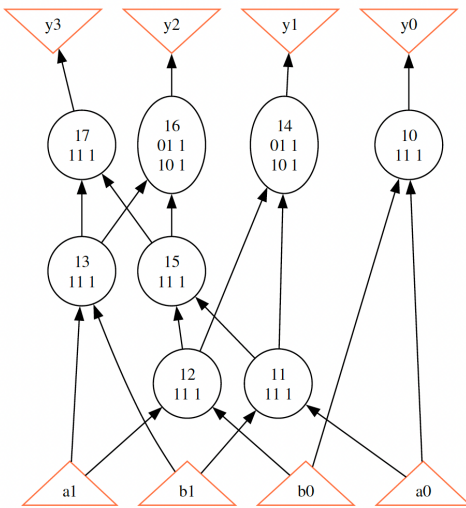
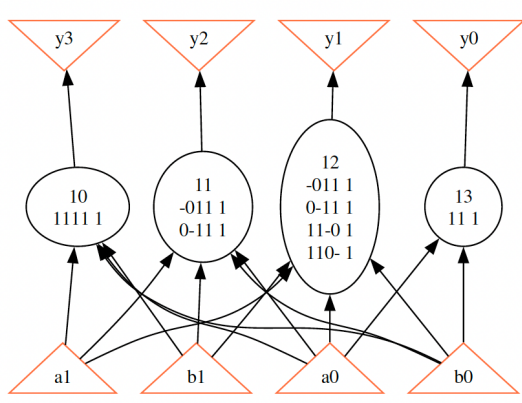
左圖 AIG 中的節點 14 和節點 16 的 XOR 閘，在 STRASH 中被進一步拆開成 18, 19, 20 和 15, 16, 17 的 AND 閘和反向邊。注意到左圖並非真正的全域 AIG，因為他仍保留原始邏輯網路(Ex: XOR)；右圖是全域 AIG，並且沒有重複的邏輯結構。

2.

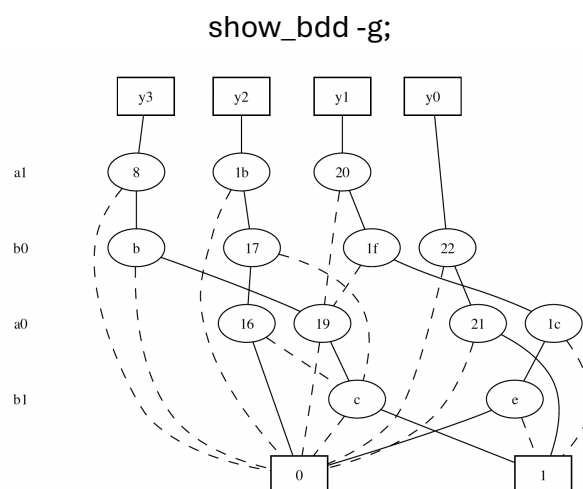
```

abc 01> r lsv/pa1/mul.blif;
Hierarchy reader flattened 2 instances of logic boxes and left 0 black boxes.
abc 02> bdd; ps; show;
mul:                               : i/o = 4/ 4 lat = 0 nd = 8 edge = 16 bdd = 16 lev = 3
abc 02> collapse; ps; show;
mul:                               : i/o = 4/ 4 lat = 0 nd = 4 edge = 14 bdd = 14 lev = 1

```

bdd	collapse
<pre> abc 02> bdd; ps; show; mul: i/o = 4/ 4 lat = 0 nd = 8 edge = 16 bdd = 16 lev = 3 </pre>	<pre> abc 02> collapse; ps; show; mul: i/o = 4/ 4 lat = 0 nd = 4 edge = 14 bdd = 14 lev = 1 </pre>
The network contains 8 logic nodes and 0 latches. 	The network contains 4 logic nodes and 0 latches. 

左圖 BDD 是原始的邏輯網路，每條邊以一個 BDD 表示；右圖 COLLAPSE 將中間邏輯壓縮為一層，並用 PIs 作為輸入、POs 作為輸出，因此所有輸出可以繪製為全域共用的 ROBDD (如 show_bdd -g; 附圖所示)。可以看到壓縮後節點 12 邏輯變得複雜許多。



3. (b)

Given a structurally hashed AIG (STRASH), convert it to a logic network with node function expressed in SOP:

```
(locally) abc 01> logic; ps; pf; show;
```

```
(globally) abc 01> collapse; sop; ps; pf; show;
```

Set up for the two-bit unsigned multiplier example:

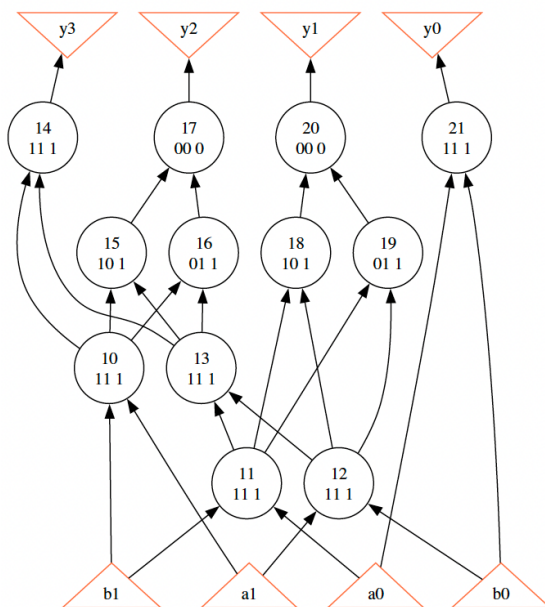
```
abc 01> r lsv/pa1/mul.blif; strash; ps;
```

1. Convert command (locally):

```
abc 03> logic; ps; pf; show;
```

```
abc 01> r lsv/pa1/mul.blif; strash; ps;
Hierarchy reader flattened 2 instances of logic boxes and left 0 black boxes.
mul      : i/o = 4/ 4 lat = 0 and = 12 lev = 4
abc 03> logic; ps; pf; show;
mul      : i/o = 4/ 4 lat = 0 nd = 12 edge = 24 cube = 12 lev = 4
n9 = (a1b1)
n10 = (a0b1)
n11 = (a1b0)
n12 = (n10n11)
n13 = (n9n12)
n14 = (n9!n12)
n15 = (!n9n12)
n16 = (n14 + n15)
n17 = (n10!n11)
n18 = (!n10n11)
n19 = (n17 + n18)
n20 = (a0b0)
```

The network contains 12 logic nodes and 0 latches.



2. Convert command (globally):

```
abc 01> collapse; sop; ps; pf; show;
```

```
abc 01> r lsv/pa1/mul.blif; strash; ps;
Hierarchy reader flattened 2 instances of logic boxes and left 0 black boxes.
mul      : i/o = 4/ 4 lat = 0 and = 12 lev = 4
abc 03> collapse; sop; ps; pf; show;
mul      : i/o = 4/ 4 lat = 0 nd = 4 edge = 14 cube = 8 lev = 1
n9 = ((a1b0)(a0b1))
n10 = (a1(b1(!a0 + !b0)))
n11 = ((a1(b0(!a0 + !b1))) + (a0(b1(!a1 + !b0))))
n12 = (b0a0)
```

The network contains 4 logic nodes and 0 latches.

